MASTER QA LEARNING PROGRAM

Phase 3: QA Engineer — Level 2 Detailed Guide

Agile, API, Database, Metrics, Risk, UAT and Advanced Testing

8 Chapters | 4–6 Weeks | Intermediate Level

(Update this Table of Contents in Word: References > Update Table)

Learning Objectives

1. Apply Agile testing practices within Scrum sprints including sprint planning, daily standups, and retrospectives.
2. Design and execute API tests using Postman and REST-based testing techniques.
3. Perform database testing with SQL to validate data integrity, referential integrity, and stored procedures.
4. Build and interpret QA dashboards with test metrics including defect density, test coverage, and pass/fail rates.
5. Apply Risk-Based Testing to prioritise test effort and manage test scope in time-constrained projects.
6. Plan and coordinate User Acceptance Testing with stakeholders and manage the sign-off process.
7. Conduct advanced exploratory testing using session-based test management and heuristic analysis.
8. Design and execute cross-browser, cross-platform, and compatibility test strategies.

Phase Duration

Phase 3 is designed for 4–6 weeks of focused self-study. It assumes you have completed Phase 2 (Level 1 Junior QA Engineer) or have equivalent hands-on experience in manual testing, defect management, and STLC.

Chapter 1 - Agile Testing in Practice

↑ Back to Index

Agile transformed software delivery — and with it, the role of QA. In a traditional Waterfall project, testing happens after development is complete. In Agile, testing is woven into every sprint. QA engineers in Agile teams are not gatekeepers at the end of a pipeline; they are quality advocates embedded in the team from day one.

At Level 2, you are expected to be a full participant in Agile ceremonies, to test features within the sprint they are built, and to provide fast, actionable feedback that keeps the team moving. This chapter gives you the vocabulary, techniques, and mindset to thrive in an Agile environment.

1.1 Scrum Fundamentals for QA

Scrum is the most widely used Agile framework. As a QA Engineer in a Scrum team, you operate in fixed-length iterations called sprints, typically lasting 1–4 weeks. Understanding the structure helps you fit your work into the team's rhythm.

Scrum Event	Duration	QA's Role
Sprint Planning	2–4 hours	Review user stories, raise testability concerns, estimate QA effort, define acceptance criteria
Daily Standup	15 minutes	Report testing progress, flag blockers, highlight dependency risks
Sprint Review	1–2 hours	Demo tested features, report test coverage and defect status to stakeholders
Sprint Retrospective	1–2 hours	Share process improvement ideas, identify recurring quality issues
Backlog Refinement	1–2 hours	Review upcoming stories, assess testability, clarify ambiguities before sprint start

1.2 User Story Validation and Three Amigos

Before a user story enters a sprint, QA should participate in the Three Amigos meeting — a structured conversation between the Product Owner (PO), Developer, and QA. The goal is to ensure everyone has the same understanding of what needs to be built and how it will be verified.

- The Product Owner clarifies the business intent: what problem does this solve for the user?

- The Developer explains how they plan to implement it: what are the technical constraints and decisions?
- The QA Engineer asks: how will we know it works? What are the edge cases, negative scenarios, and integration points?
- The output is a shared set of acceptance criteria and a preliminary list of test scenarios.

How It Works Internally

Three Amigos meetings typically last 15–30 minutes per story. The best QA engineers come prepared with a list of questions and example test scenarios drafted in advance. Tools like Gherkin (Given/When/Then) are often used to write acceptance criteria in a format that is both readable by the business and automatable.

1.3 Acceptance Criteria and Definition of Done

Acceptance criteria define the specific conditions that must be met for a user story to be considered complete. The Definition of Done (DoD) is the team's shared checklist of what 'done' means across all stories.

Acceptance Criteria (Story-specific)	Definition of Done (Team-wide)
Login must work with valid credentials	Unit tests written and passing
Error message shown for invalid password	Code reviewed and merged to main
Session expires after 30 minutes of inactivity	QA signed off with all test cases passed
Remember Me checkbox persists session for 7 days	No P1/P2 defects open
Password field masked on screen	Deployed to staging environment

1.4 Sprint Testing Strategy

Within each sprint, QA must balance exploratory testing of new features with regression testing of existing functionality. A structured approach prevents both over-testing (slowing the team) and under-testing (releasing defects).

1. Day 1-2: Participate in sprint planning. Review all stories and create a sprint test plan.
2. Day 2-3: As developers complete stories, begin testing immediately — don't wait for all stories to be done.
3. Day 3-7: Execute test cases, raise defects, retest fixes. Track progress on the test dashboard.
4. Day 7-8: Run regression suite on critical paths. Prepare sprint review demo.
5. Day 9-10: Sprint closure — confirm DoD, update test artefacts, raise carry-overs for next sprint.

In Practice

In fast-paced Agile teams, QA is often the bottleneck if testing only happens at the end of the sprint. Shift left by reviewing stories on Day 1, creating test cases on Day 2-3, and starting testing as soon as the first story is ready — even if other stories are still in development.

1.5 Test Coverage in Agile

Unlike Waterfall, where test coverage targets might be set months in advance, Agile requires adaptive coverage decisions at the story level. Focus on covering the happy path, primary alternate flows, and the most likely failure modes first — then expand if time permits.

- Functional coverage: Does the feature do what the acceptance criteria say?
- Boundary coverage: What happens at the edges of valid input ranges?
- Negative coverage: What happens with invalid data, missing fields, or unauthorized access?
- Integration coverage: Does this feature work correctly with related features and services?
- Regression coverage: Does this change break any existing functionality?

1.6 Agile Metrics for QA

Metric	Formula / Definition	What it Tells You
Sprint Velocity	Story points completed per sprint	Team capacity and predictability
Defect Injection Rate	Defects found / story points completed	Code quality per sprint
Escaped Defects	Defects found after sprint close / total defects	Effectiveness of sprint testing
Test Case Execution Rate	% of planned test cases executed	QA progress within sprint
Defect Resolution Time	Avg hours from defect logged to fixed	Developer responsiveness

1.7 Best Practices

- Treat test cases as living documents — update them every sprint as the product evolves.
- Never carry untested stories into the next sprint; it creates technical quality debt.
- Build relationships with developers — pair testing (developer + QA sit together) dramatically reduces defect cycle time.
- Use a sprint test summary template to communicate quality status consistently at every sprint review.
- Automate the regression suite incrementally — pick the 20% of tests that catch 80% of regressions first.

1.8 Common Mistakes

- Starting testing only on the last two days of the sprint — this creates panic, incomplete coverage, and pressure to cut testing.
- Not attending Three Amigos — you lose the chance to catch ambiguities before code is written.
- Testing only happy paths — Agile moves fast, making it easy to skip edge cases; document them even if you don't test them all.
- Ignoring regression — new stories can break old features; regression is not optional, it is mandatory.

Warning

Velocity pressure is real in Agile. Teams sometimes push QA to approve stories that are not fully tested to meet sprint goals. Resist this. An untested story that reaches production creates far more disruption than a missed sprint commitment.

1.9 Exercises

6. Take a user story from any product you use daily (e.g. a food delivery app) and write 10 acceptance criteria covering happy path, alternate flows, and edge cases.
7. Simulate a Three Amigos meeting with two colleagues. Each person takes one role: PO, Dev, QA. Spend 20 minutes discussing one user story and document the output.
8. Create a sprint test plan template for a 2-week sprint covering 5 stories. Include columns for story ID, test approach, test cases, priority, and status.
9. Calculate the escaped defect rate for a fictional sprint where 30 defects were found in sprint and 8 were found post-release.
10. Draw a sprint timeline showing when QA activities happen relative to development activities across a 10-day sprint.

Quick Quiz

1. What are the five Scrum ceremonies and what is QA's role in each?
2. What is the Three Amigos meeting and why is it critical for QA?
3. What is the difference between acceptance criteria and the Definition of Done?
4. How would you handle a situation where all stories are handed to QA on the last day of the sprint?
5. What does escaped defect rate measure and why does it matter?

Interview Questions

1. How do you manage testing when developers deliver all stories on the last two days of the sprint?
2. Walk me through how you would conduct a Three Amigos session for a complex user story.
3. How do you decide what to automate and what to test manually in an Agile environment?
4. Describe a situation where you caught a major issue during sprint planning or backlog refinement.

Chapter Summary

• Agile testing is continuous — QA starts on Day 1 of the sprint, not after development is complete.

• Three Amigos meetings align PO, Dev, and QA on acceptance criteria before a single line of code is written.

- Sprint testing requires balancing new feature testing with regression of existing functionality.
- Key Agile QA metrics include defect injection rate, escaped defect rate, and test execution rate.
- Agile QA engineers are quality advocates embedded in the team, not gatekeepers at the end of the pipeline.

Chapter 2 - API Testing with Postman

↑ Back to Index

Modern software applications are built as networks of services communicating through APIs. As a QA Engineer at Level 2, you need to go beyond the user interface and test the business logic at the API layer. API testing is faster than UI testing, catches defects earlier, and can be automated more reliably.

Postman is the industry-standard tool for API testing. It allows you to send HTTP requests, inspect responses, write assertions, and build automated collections. This chapter teaches you to design, execute, and structure professional API test suites.

2.1 REST API Fundamentals

REST (Representational State Transfer) is the dominant architectural style for web APIs. Before testing, you need to understand the building blocks of every API request and response.

HTTP Method	Operation	QA Test Focus
GET	Retrieve a resource	Response code 200, correct data returned, empty list vs null
POST	Create a new resource	201 created, location header, duplicate prevention, invalid payload
PUT	Replace a resource fully	200/204, full replacement vs partial update, non-existent resource
PATCH	Partially update a resource	Partial update fields, unchanged fields remain intact
DELETE	Remove a resource	204 no content, 404 on second delete, cascading deletes

2.2 Anatomy of an API Request and Response

Every API test involves crafting a request and verifying the response. Understanding each component is essential to thorough testing.

How It Works Internally

When you send an API request, it travels over HTTP/HTTPS to a web server. The server routes the request to the correct endpoint handler (often a REST controller), processes business logic, queries a database, and returns a JSON or XML response with a status code, headers, and body. QA validates all three layers: status code (did it succeed?), headers (content type, auth tokens), and body (correct data?).

Component	Example	What to Test
URL / Endpoint	https://api.example.com/v1/users/42	Valid/invalid IDs, versioning, special characters

HTTP Method	GET, POST, PUT, DELETE	Wrong method returns 405 Method Not Allowed
Headers	Authorization: Bearer <token>	Missing auth → 401, expired token → 401/403
Query Params	?page=1&limit=20&sort=name	Pagination, filtering, sorting, invalid params
Request Body	{ "name": "John", "email": "j@x.com" }	Required fields, field types, max length, XSS injection
Status Code	200 OK, 201 Created, 400 Bad Request	Each scenario maps to a specific expected code
Response Body	{ "id": 42, "name": "John" }	Field presence, data types, values, null handling
Response Headers	Content-Type: application/json	Correct content type, CORS headers, caching

2.3 Setting Up Postman for API Testing

Postman organises your work into Collections (folders of related requests), Environments (variable sets for dev/staging/prod), and Test Scripts (JavaScript assertions). Set these up before writing any tests.

11. Create a Collection named for the project (e.g. 'User Service — QA Suite').
12. Create Environments: Dev, Staging, Production. Add variables: base_url, auth_token, user_id.
13. Set the Authorization tab to Bearer Token using the {{auth_token}} variable.
14. For each endpoint, create a request folder (e.g. 'POST /users', 'GET /users/:id').
15. Write pre-request scripts to set up test data and post-request tests to assert on the response.

```
// Postman Test Script — POST /users happy path
pm.test('Status code is 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Response has user id', function () {
  const json = pm.response.json();
  pm.expect(json.id).to.be.a('number');
  pm.expect(json.id).to.be.above(0);
});

pm.test('Email matches request', function () {
  const json = pm.response.json();
  pm.expect(json.email).to.eql(pm.environment.get('test_email'));
});

// Save id for use in subsequent requests
```

```
pm.environment.set('created_user_id', pm.response.json().id);
```

2.4 API Test Design Techniques

Apply the same test design techniques from Phase 2 to API testing. The difference is that the 'input' is the request and the 'output' is the response.

Technique	API Testing Application
Boundary Value Analysis	Min/max field lengths (name: 1 char, 255 chars, 256 chars), numeric ranges
Equivalence Partitioning	Valid emails vs invalid emails vs missing email field
Negative Testing	Wrong data types (string where int expected), SQL injection in text fields
State Transition	User status flow: PENDING → ACTIVE → SUSPENDED → DELETED
End-to-End Flow	Create user → Get user → Update user → Delete user → Verify 404

In Practice

In real projects, always test with a fresh environment for each test run. Use pre-request scripts to create test data and cleanup scripts to delete it after. Never test destructive operations (DELETE, UPDATE) on production data — always use dedicated QA environments with test data.

2.5 Authentication and Authorization Testing

Security testing is a critical part of API testing. Every endpoint must be tested for both authentication (who are you?) and authorization (what are you allowed to do?).

- No token: Request without Authorization header should return 401 Unauthorized.
- Invalid token: Malformed or expired token should return 401 Unauthorized.
- Insufficient permissions: Valid token for a lower-privilege role accessing a restricted endpoint should return 403 Forbidden.
- Cross-tenant access: User from Tenant A should not access Tenant B's resources — returns 403 or 404.
- Token expiry: Verify tokens expire after the configured TTL and refresh token flow works correctly.

```
// Test: Unauthorized access
pm.test('Missing token returns 401', function () {
  // Remove Authorization header from this request
  pm.response.to.have.status(401);
});
```

```
// Test: Forbidden access (valid token, wrong role)
pm.test('Non-admin cannot delete user', function () {
  // Use a 'viewer' role token
  pm.response.to.have.status(403);
});
```

2.6 Running Collections and Generating Reports

Postman's Collection Runner lets you execute all requests in a collection sequentially. Newman is Postman's CLI runner, allowing API tests to be run from a terminal or integrated into a CI/CD pipeline.

```
# Run a Postman collection via Newman
newman run collection.json \
  --environment staging.json \
  --reporters cli,html \
  --reporter-html-export report.html

# Run with iteration data (data-driven testing)
newman run collection.json \
  --environment staging.json \
  --iteration-data test_data.csv
```

2.7 Best Practices

- Use environment variables for all URLs, tokens, and IDs — never hardcode values in requests.
- Chain requests: save the output of a POST (created resource ID) and use it in subsequent GET/PUT/DELETE tests.
- Test both the happy path and at least 3–5 negative scenarios for every endpoint.
- Include schema validation in your test scripts to catch structural changes in the response early.
- Integrate Newman into your CI/CD pipeline so API tests run automatically on every code merge.

2.8 Common Mistakes

- Testing only happy paths — APIs fail in unexpected ways; negative testing is as important as positive.
- Not testing authentication and authorization — these are the most common security vulnerabilities.
- Hardcoding URLs and tokens — makes the collection unusable across environments.
- Ignoring response headers — content-type, caching, and CORS headers are frequently misconfigured.

Warning

Never run destructive API tests (DELETE, bulk updates) against a shared QA environment without coordinating with your team. A mistakenly executed DELETE on a shared dataset can derail testing for the entire team for hours.

2.9 Exercises

16. Using Postman, create a collection against any public REST API (e.g. JSONPlaceholder at jsonplaceholder.typicode.com). Test GET /posts, POST /posts, PUT /posts/1, and DELETE /posts/1 with at least 3 assertions each.
17. Write test scripts that chain requests: create a resource, retrieve it by ID, update it, then delete it. Verify each step.
18. Test the authentication flow of any API: test with no token, an invalid token, and a valid token. Verify correct HTTP status codes for each case.
19. Export your Postman collection and run it using Newman from the command line. Generate an HTML report.
20. Design a test matrix for a user registration API with fields: email, password, name, phone. Apply BVA and EP to create at least 15 test cases.

Quick Quiz

1. What is the difference between a 401 and a 403 HTTP response status code?
2. How do you test pagination in a REST API that returns paginated results?
3. What is Newman and how does it extend Postman's capabilities?
4. Why should you use environment variables instead of hardcoding values in Postman?
5. How would you test that a DELETE endpoint correctly returns 404 on a second call for the same resource?

Interview Questions

1. Walk me through how you would test a new POST /orders API endpoint end to end.
2. How do you handle authentication in your API test collections?
3. Describe how you would integrate API tests into a CI/CD pipeline.
4. How would you test that an API correctly enforces role-based access control?

Chapter Summary

• API testing validates business logic faster and more reliably than UI testing.
• Postman organises tests into Collections, Environments, and Test Scripts with JavaScript assertions.
• Every endpoint must be tested for authentication, authorization, and both positive and negative scenarios.
• Newman enables API tests to run in CI/CD pipelines for continuous quality feedback.
• Chain requests using environment variables to create realistic end-to-end API test flows.

Chapter 3 - Database Testing and SQL for QA

↑ Back to Index

Every application stores data somewhere. When a user places an order, registers an account, or uploads a document, that data lands in a database. UI and API tests verify that the application behaves correctly — but database testing verifies that the data itself is correct, complete, and consistent. A UI test might say the order was placed successfully. A database test verifies the order actually exists with the right amount, product IDs, and customer reference.

As a Level 2 QA Engineer, you are expected to write and run SQL queries to verify application behaviour at the data layer. This chapter gives you the SQL knowledge and testing techniques to do that confidently.

3.1 Why Database Testing Matters

Scenario	UI/API Test Result	What Database Testing Reveals
User registration	Success message shown	Email not stored, or stored with wrong encoding
Order placement	Order confirmation displayed	Inventory not decremented, duplicate order record created
Password change	Confirmation email sent	Old password still valid in database — security bug
Data deletion	Record removed from screen	Soft delete flag not set; data still exposed in reports
Price calculation	Correct total shown on screen	Rounding error stored in DB — affects finance reporting

3.2 SQL Fundamentals for QA Engineers

You need four SQL skills for effective database testing: selecting data, filtering it, joining related tables, and counting/aggregating results.

```
-- SELECT: Retrieve data from a table
SELECT id, email, created_at FROM users WHERE status = 'ACTIVE';

-- Filter with multiple conditions
SELECT * FROM orders
WHERE customer_id = 1001
  AND status = 'PLACED'
  AND created_at >= '2026-01-01';

-- JOIN: Combine related tables
SELECT o.id AS order_id, u.email, o.total_amount
FROM orders o
```

```
INNER JOIN users u ON o.customer_id = u.id
WHERE o.status = 'PLACED';

-- Aggregate: Count, sum, group
SELECT status, COUNT(*) AS total, SUM(total_amount) AS revenue
FROM orders
GROUP BY status;

-- Subquery: Find users with no orders
SELECT * FROM users
WHERE id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

How It Works Internally

Relational databases like MySQL, PostgreSQL, and SQL Server store data in tables with rows and columns. Tables are related through foreign keys. When you write JOIN queries, the database engine looks up matching rows across tables using index structures (B-trees) for performance. As a QA engineer, you read data — you rarely modify it directly. Always use SELECT statements for verification. INSERT/UPDATE/DELETE in QA is only for setting up test data, never for fixing production issues.

3.3 Data Integrity Testing

Data integrity ensures the data in the database is accurate, complete, and consistent. There are four types of integrity to test:

Integrity Type	Definition	SQL Test Example
Entity Integrity	Every row has a unique, non-null primary key	SELECT COUNT(*) FROM users WHERE id IS NULL
Referential Integrity	Foreign keys reference valid rows in parent tables	SELECT * FROM orders WHERE customer_id NOT IN (SELECT id FROM users)
Domain Integrity	Data values are within defined valid ranges and types	SELECT * FROM products WHERE price < 0 OR price IS NULL
User-Defined Integrity	Business rules enforced in DB (constraints, triggers)	SELECT * FROM orders WHERE quantity > stock_available

3.4 Test Scenarios for Common Database Operations

For each application feature that stores data, design SQL-based verification queries alongside your functional test cases.

- Create operation: After a UI/API create action, verify the new row exists with correct field values, auto-generated fields (ID, timestamps), and default values.

- Update operation: Verify only the target record changed; surrounding records are untouched; updated_at timestamp was updated.
- Delete operation: For soft deletes, verify the deleted_at flag is set and status changed; for hard deletes, verify the row is completely gone.
- Bulk operations: For imports or batch jobs, verify row count matches expectation, no duplicates were created, and partial failures rolled back correctly.
- Cascading rules: When a parent record is deleted, verify child records are either deleted (cascade) or prevented (restrict) per the business rule.

```
-- Verify create: user was stored correctly
SELECT id, email, status, created_at
FROM users
WHERE email = 'testuser@example.com'
  AND status = 'PENDING';

-- Verify update: only target record changed
SELECT id, name, updated_at FROM products WHERE id = 42;
-- Also check a nearby record was not changed:
SELECT id, name, updated_at FROM products WHERE id = 43;

-- Verify soft delete: deleted_at populated, status changed
SELECT id, status, deleted_at FROM users WHERE id = 101;
-- Expected: status='DELETED', deleted_at IS NOT NULL
```

In Practice

Always run database verification queries as part of your test case steps, not as an afterthought. For manual tests: run the UI action, then immediately query the database to confirm the data. For automated tests: add a database assertion step after the API/UI step. This is the difference between testing that an action succeeds and verifying its outcome.

3.5 Stored Procedure and Trigger Testing

Stored procedures are pre-written SQL programs that execute business logic inside the database. Triggers are automatic procedures that fire on INSERT, UPDATE, or DELETE events. Both need dedicated testing.

- Test stored procedures by calling them with valid inputs and verifying the output and side effects.
- Test boundary inputs and null/missing parameters — stored procedures often lack the validation that application code has.
- For triggers, verify they fire correctly on the intended events and do not fire on unintended events.
- Test trigger failure scenarios — what happens if a trigger fails? Does the parent transaction roll back?
- Always test stored procedures and triggers in isolation before testing them through the full application stack.

3.6 Performance and Volume Testing at the DB Layer

A query that takes 50ms with 1,000 rows might take 60 seconds with 10 million rows. Volume testing the database layer catches performance issues before they reach production.

- Check query execution plans with EXPLAIN or EXPLAIN ANALYZE to verify indexes are being used.
- Test with data volumes representative of production — don't test with 100 rows when production has 50 million.
- Identify queries without WHERE clauses on indexed columns — these cause full table scans.
- Test concurrent inserts and updates for deadlock and locking issues under load.

3.7 Best Practices

- Always use SELECT for verification — never modify production data directly with UPDATE or DELETE.
- Keep a library of reusable QA SQL queries for your project — it saves time and ensures consistency.
- Check NULL handling explicitly — many bugs come from unexpected NULL values in calculations and joins.
- Use transactions for test data setup so you can roll back cleanly after each test run.
- Document the database schema for your application — understand table relationships before writing tests.

3.8 Common Mistakes

- Only testing through the UI without verifying database state — lets data bugs slip through undetected.
- Forgetting to check foreign key relationships after create/delete operations.
- Ignoring NULL values in test data — NULLs break calculations and JOIN results in subtle ways.
- Running UPDATE or DELETE queries accidentally on production — always double check your WHERE clause.

Warning

Before running any SQL against a database, verify you are connected to the correct environment. Many experienced QA engineers have accidentally modified a production database by running a script against the wrong connection. Keep production and QA connections clearly labelled and separate.

3.9 Exercises

21. Set up a local MySQL or PostgreSQL database. Create a users table and an orders table with a foreign key relationship. Insert 20 rows of test data.
22. Write queries to: find all users with no orders, find orders with a total above 1000, count orders by status, find duplicate email addresses.

23. Design 10 database test cases for a 'create user' feature. Include verification of all stored fields, auto-generated values, and default values.
24. Write a SQL query to detect orphaned records — orders that reference a user ID that does not exist in the users table.
25. Practice referential integrity testing: try to INSERT an order with a non-existent customer_id and verify the database constraint rejects it.

Quick Quiz

1. What are the four types of data integrity and how would you test each one?
2. What is the difference between a hard delete and a soft delete? How do you verify each in the database?
3. Why should QA engineers use SELECT statements rather than UPDATE or DELETE when verifying data?
4. How would you test a stored procedure that calculates order discounts?
5. What is a foreign key and why is referential integrity testing important?

Interview Questions

1. Walk me through how you would verify that a user registration feature correctly stored all required data.
2. How would you test a batch data import process that processes 100,000 records nightly?
3. Describe a database bug you found (or hypothetically would look for) that a UI test would not catch.
4. How do you handle test data setup and teardown for database testing?

Chapter Summary

• Database testing verifies that application data is stored correctly, completely, and consistently.
• Four SQL skills are essential: SELECT with filtering, JOIN for related tables, aggregation, and subqueries.
• Test all four data integrity types: entity, referential, domain, and user-defined.
• Always verify database state after UI/API actions — the screen may show success while the database holds an error.
• Use SELECT for verification only; never modify data directly in production or shared environments.

Chapter 4 - Test Metrics, Dashboards and Reporting

↑ Back to Index

Numbers tell stories. A QA Engineer who says 'testing is going well' provides no useful information to the team. A QA Engineer who says 'we have executed 340 of 420 planned test cases, found 23 defects of which 18 are resolved, and 2 P1 defects remain open — we recommend delaying release by 48 hours' is indispensable. Metrics transform QA from a cost centre into a strategic function.

At Level 2, you are responsible for tracking your own test progress and reporting it accurately to the team and stakeholders. This chapter teaches you which metrics to collect, how to calculate them, and how to communicate them clearly.

4.1 Core Test Execution Metrics

Metric	Formula	Target / Interpretation
Test Case Execution Rate	Executed / Planned × 100	> 95% by end of sprint/cycle
Test Pass Rate	Passed / Executed × 100	> 90% for go/no-go decision
Test Fail Rate	Failed / Executed × 100	< 10% for low-risk release
Blocked Test Cases	Count of tests blocked by defects	0 blocked is ideal; track and escalate
Test Coverage	Features tested / Total features × 100	100% coverage of P1/P2 features mandatory

4.2 Defect Metrics

Metric	Formula	Why It Matters
Defect Density	Defects / Test cases × 100	Higher density signals unstable modules
Defect Removal Efficiency (DRE)	Defects found in QA / Total defects × 100	Measures how well QA catches defects before production
Escaped Defect Rate	Production defects / Total defects × 100	Low % = effective QA; high % = testing gaps
Defect Age	Days from logged to resolved	Long age signals process or communication issues
Defect Reopen Rate	Reopened defects / Total closed defects × 100	High rate signals poor fix quality or unclear descriptions

How It Works Internally

DRE (Defect Removal Efficiency) is the gold-standard QA health metric for many enterprise clients. A DRE of 95% means 95% of all defects were found by QA before reaching production. The remaining 5% escaped to production. Enterprise SLAs often require DRE > 95% as a contractual commitment. Tracking this requires logging ALL defects — including those found and fixed within the sprint.

4.3 Building a Test Status Report

A good test status report answers four questions for every stakeholder: What was planned? What did we do? What did we find? What do we recommend? Keep it concise, visual, and honest.

Section	Content	Audience
Executive Summary	2-3 lines: overall quality status, go/no-go signal	Management, Product Owner
Test Execution Summary	Planned vs executed vs passed/failed table	Scrum Master, Delivery Manager
Defect Summary	Total open/closed by severity and priority	Development Lead, PO
Risk and Blockers	Open risks, blocked tests, missing environments	Scrum Master, Tech Lead
Recommendation	Go / Conditional Go / No-Go with rationale	All stakeholders

4.4 Daily Status Update Template

In an Agile sprint, QA provides a daily status update — typically in the standup and via a shared dashboard. Keep it brief and factual.

Daily QA Status Format
Sprint Day X of Y
Planned: 120 test cases | Executed: 87 (73%) | Passed: 79 | Failed: 8
Defects: 12 open (P1: 1, P2: 4, P3: 7) | 3 resolved today
Blockers: LOGIN-234 blocking 5 test cases — waiting on backend fix
Focus today: Complete checkout flow testing (20 cases remaining)

4.5 Quality Gates and Go/No-Go Criteria

A quality gate is a set of measurable criteria that must be met before releasing software. Defining these upfront prevents subjective go/no-go debates at release time.

Gate Criterion	Typical Threshold	Rationale
Test Execution Rate	>= 95%	All planned tests must be executed

Gate Criterion	Typical Threshold	Rationale
Test Pass Rate	>= 90%	Most tests must pass
P1 Defects Open	= 0	No showstopper defects at release
P2 Defects Open	<= 2 with workaround	Minor blockers acceptable if workaround exists
Regression Pass Rate	>= 100% of critical path	No regressions in business-critical flows

In Practice

Document your quality gates in the test plan at the start of the project — not at release time. Stakeholders are far more likely to accept a delay when the quality gates were agreed upfront. 'We agreed before the project started that zero P1 defects was required for release' is a much stronger position than trying to argue the case at a release meeting.

4.6 Metrics to Avoid

Some QA metrics create perverse incentives and should be avoided or used cautiously.

- Number of test cases executed: Creates incentive to execute tests faster, not better. Focus on pass/fail rates instead.
- Number of defects found: Can be gamed by logging trivial defects; focus on severity-weighted defect metrics.
- 100% pass rate before release: Unsustainable and often achieved by deleting or skipping failing tests.
- Defects-per-tester: Damages team collaboration and creates adversarial dynamics between QA and development.

4.7 Best Practices

- Update metrics daily — stale metrics are useless and damage your credibility.
- Use a shared dashboard (JIRA, Confluence, or a shared spreadsheet) visible to the entire team.
- Always include a recommendation in your report — stakeholders expect QA to have a view on release readiness.
- Track metrics over multiple sprints to identify trends — a rising defect density often predicts a quality crisis.
- Keep historical data — before/after comparisons of defect density, DRE, and escape rate are powerful for process improvement discussions.

4.8 Common Mistakes

- Reporting only positive metrics and hiding the bad news — this destroys trust and leads to surprises at release.
- Not reporting blockers clearly — if 30 test cases are blocked, the team lead needs to know immediately, not at the end of the sprint.
- Tracking too many metrics — focus on 5–7 key metrics; metric overload obscures what matters.
- Calculating metrics from memory rather than from tool data — always pull numbers from JIRA, TestRail, or your test management tool.

Warning

Never falsify or massage metrics to look better. QA credibility depends entirely on the accuracy and honesty of your reporting. A test pass rate that is 80% and honestly reported is far more valuable than a claimed 98% that masks hidden risk.

4.9 Exercises

26. Given: 200 planned tests, 185 executed, 162 passed, 23 failed, 8 blocked. Calculate: execution rate, pass rate, fail rate, and write a 3-sentence executive summary.
27. In a release, QA found 45 defects. After release, 3 additional defects were found in production. Calculate DRE and escaped defect rate.
28. Design quality gate criteria for a mobile banking app release. Define thresholds for all major metrics and justify each threshold.
29. Create a daily test status report template in Excel or Google Sheets that auto-calculates execution rate, pass rate, and defect metrics from raw input data.
30. Analyse a trend: in Sprint 1, defect density was 5%. In Sprint 2, it rose to 8%. In Sprint 3, it rose to 12%. Write a paragraph explaining what this trend suggests and what actions you would recommend.

Quick Quiz

1. What is Defect Removal Efficiency and why is it important for enterprise clients?
2. What are quality gates and why should they be defined before a project starts?
3. A team has 80% pass rate and 2 open P1 defects. Should they release? Justify your answer.
4. Why is 'number of test cases executed' considered a poor QA metric?
5. What is the difference between test coverage and test execution rate?

Interview Questions

1. How do you measure the effectiveness of your QA process?
2. Describe how you would set up a QA dashboard for a project from scratch.
3. How would you communicate a recommendation to delay a release to senior stakeholders?
4. What metrics do you track daily, and how do you present them to your team?

Chapter Summary

• QA metrics transform testing from a subjective activity into a measurable, manageable process.
• Core metrics: execution rate, pass rate, defect density, DRE, and escaped defect rate.

- Quality gates should be defined at project start — not at release time.
- Daily status updates keep the team informed and allow early intervention when quality degrades.
- Honest reporting, even when the news is bad, is the foundation of QA credibility.

Chapter 5 - Risk-Based Testing and Test Prioritisation

↑ Back to Index

You will never have enough time to test everything. Every project operates under time and resource constraints. Risk-Based Testing (RBT) is the disciplined approach to making smart decisions about where to focus your testing effort to maximise quality coverage given limited time.

RBT is not about cutting testing. It is about prioritising intelligently so that the highest-risk areas receive the most thorough testing and lower-risk areas are tested proportionally less. Done well, RBT delivers better quality outcomes than untargeted full coverage.

5.1 Understanding Risk in Software Testing

In QA, risk is the combination of the likelihood that something will fail and the impact of that failure if it does. Every area of a system carries some level of risk. Your job is to identify, assess, and mitigate that risk through targeted testing.

Risk Factor	Examples	Questions to Ask
Likelihood of Failure	New code, complex logic, external dependencies	How much has this changed? How complex is it? Has it failed before?
Business Impact	Payment processing, login, data export	What happens to users if this fails? What is the revenue/reputational impact?
Frequency of Use	Homepage, core workflows vs admin settings	How many users will be affected if this breaks?
Regulatory Risk	GDPR, PCI-DSS, HIPAA compliance	Does a failure here create legal or compliance exposure?
Integration Risk	Third-party APIs, payment gateways, bank connections	Do external dependencies have known reliability issues?

5.2 The Risk Matrix

A risk matrix maps each feature or test area against likelihood of failure (x-axis) and impact of failure (y-axis). This gives you a visual prioritisation tool.

How It Works Internally

Risk scoring: assign each area a Likelihood score (1-5) and an Impact score (1-5). Multiply them to get a Risk Priority Number (RPN): RPN = Likelihood × Impact. Areas with RPN 16-25 are Critical (test first, most deeply). RPN 9-15 are High (thorough testing). RPN 4-8 are Medium (standard testing). RPN 1-3 are Low (smoke test only).

Feature Area	Likelihood (1-5)	Impact (1-5)	RPN	Test Priority
Payment processing	3	5	15	High — full suite

User login/logout	2	5	10	High — full suite
New checkout redesign	4	4	16	Critical — full suite + exploratory
Product search	2	3	6	Medium — standard
Admin theme settings	1	1	1	Low — smoke only

5.3 Identifying Risks in a Sprint

Risk identification should happen at the start of every sprint during planning and Three Amigos. Ask these questions for each user story:

31. Is this new functionality or a change to existing functionality? (Changed code = higher risk)
32. How complex is the implementation? (Complex = higher risk)
33. How many other features does this integrate with? (High integration = higher risk)
34. Has this area been a source of bugs in previous sprints? (History = higher risk)
35. How critical is this feature to the business? (Core workflow = higher impact)
36. Is this area covered by an automated regression suite? (No automation = higher residual risk)

In Practice

In real projects, you rarely have time for formal risk matrices at story level. Instead, develop an intuition: new = risky, complex = risky, payment = high impact, rarely used admin = low impact. Over time, you build a mental model of your application's risk topology. The formal matrix is most valuable at release time when deciding where to focus your final regression pass.

5.4 Test Prioritisation Strategies

- Priority 1 — Critical path: The minimum set of tests that cover the most essential user journeys. If these fail, the release is blocked.
- Priority 2 — High risk: Features with high RPN scores, recently changed code, complex business logic.
- Priority 3 — Regression: Standard regression suite for stable, previously tested functionality.
- Priority 4 — Low risk: Rarely used features, cosmetic changes, admin tools used by a handful of users.
- Defer: Features with very low RPN that are completely unchanged from last release.

5.5 Risk-Based Test Planning

Document your risk assessment in the test plan. Stakeholders need to understand what you are testing, what you are not testing, and why. Explicitly acknowledged residual risk is a business decision — implicit untested risk is a QA failure.

Test Plan Section	Risk-Based Content
Scope — In	Features tested: payment, login, checkout, product search (all P1/P2 features)
Scope — Out	Features not tested: admin theme settings, report export (no changes, low risk, deferred)
Risk Register	Identified risks with likelihood, impact, RPN scores, and mitigation strategy
Test Prioritisation	Priority 1 tests must pass before P2/P3 are executed
Residual Risk Acknowledgement	Risk owner (product owner) signs off on untested areas

5.6 Best Practices

- Re-assess risk after each sprint — the risk profile changes as code changes.
- Track which test areas produce the most defects — this becomes your empirical risk data for future sprints.
- Always document out-of-scope items and get sign-off from the product owner — this protects QA if an untested area breaks in production.
- Use code coverage reports from developers to identify areas with low unit test coverage — these are higher QA risk.
- In very time-constrained sprints, use a 'sanity pass' for low-priority areas: one representative test per feature rather than the full suite.

5.7 Common Mistakes

- Applying equal effort to all features regardless of risk — wastes time on low-risk areas while high-risk areas are under-tested.
- Not documenting what you are NOT testing — creates liability when out-of-scope areas break.
- Ignoring history — if a module has produced 30% of all defects over the last 6 sprints, it is high-risk regardless of how simple the change looks.
- Letting developers define test priority — risk assessment requires a QA and business perspective, not just a technical one.

Warning

RBT is not a licence to skip testing. Stakeholders must explicitly accept the risk of untested areas in writing (email or JIRA comment is fine). If an untested area breaks in production and there is no sign-off, QA will be blamed. Document the risk, get the sign-off, protect yourself.

5.8 Exercises

37. For an e-commerce application, identify 10 feature areas and assign each a Likelihood (1-5) and Impact (1-5) score. Calculate the RPN and rank the areas by test priority.
38. Write a test plan scope section for a 2-week sprint with 12 features but enough capacity for only 8. Explicitly document what is in scope, out of scope, and why.
39. A sprint finishes with 15% of planned tests unexecuted due to blocked defects. Write a release recommendation addressing the tested areas, untested areas, known risks, and a recommended decision.
40. Create a risk register template with columns for: Feature, Change Description, Likelihood, Impact, RPN, Test Priority, Mitigation Strategy, and Risk Owner.
41. Review the defect history of any project you have worked on. Identify which 20% of the codebase produced 80% of the defects. How does this change your risk-based test priorities?

Quick Quiz

1. What is Risk Priority Number (RPN) and how is it calculated?
2. What is the difference between risk likelihood and risk impact? Give an example of each.
3. How would you handle a situation where there is only time to execute 60% of planned tests before a release?
4. Why is it important to document out-of-scope test areas and get stakeholder sign-off?
5. How does defect history from previous sprints inform risk-based test prioritisation?

Interview Questions

1. How do you prioritise your testing when you have half the time you need?
2. Describe how you would perform risk-based testing on a major release with 50 features.
3. How do you communicate residual risk and out-of-scope testing to stakeholders?
4. Tell me about a time when risk-based testing helped you catch a critical defect that might have been missed.

Chapter Summary

- Risk-Based Testing prioritises test effort by combining likelihood of failure and business impact.
- The Risk Priority Number (RPN = Likelihood × Impact) gives each area a quantified priority score.
- Risk assessment should happen at sprint planning, not retrospectively.
- Always document out-of-scope areas and get explicit stakeholder sign-off on residual risk.
- Defect history is the most reliable empirical data for identifying high-risk areas in future sprints.

Chapter 6 - User Acceptance Testing and Stakeholder Sign-Off

↑ Back to Index

User Acceptance Testing (UAT) is the final validation that software meets business requirements before it is released to real users. Unlike system testing — which verifies that the software works correctly — UAT verifies that the software is fit for purpose for the people who will use it. It is the bridge between technical delivery and business acceptance.

As a Level 2 QA Engineer, you may be asked to plan UAT, coordinate UAT testers (who are typically business users, not QA engineers), track UAT defects, and facilitate the sign-off process. This chapter gives you the structure to manage UAT professionally.

6.1 UAT vs System Testing — The Critical Distinction

Dimension	System Testing (QA)	User Acceptance Testing (Business)
Who tests	QA Engineers	Business users, end users, product owners
What is verified	Technical correctness — does it work as specified?	Business fitness — does it work for the user in real scenarios?
Test basis	Functional specifications, technical requirements	Business processes, real-world use cases, business rules
Environment	QA/staging environment	UAT environment (as close to production as possible)
Defect types found	Technical bugs, integration failures, edge cases	Missing functionality, workflow mismatches, usability issues
Sign-off authority	QA sign-off (technical readiness)	Business sign-off (business readiness)

6.2 Planning UAT

UAT planning should begin at least two sprints before the UAT period starts. Late UAT planning is the single biggest cause of rushed, poorly executed, and ultimately ineffective UAT.

42. Define UAT scope: Which features are in scope for UAT? What business processes will be tested?
43. Identify UAT participants: Who are the business users, product owners, and sign-off authorities?
44. Define UAT entry criteria: What must be true before UAT begins? (All P1/P2 defects resolved, stable build deployed to UAT environment, test data loaded.)
45. Define UAT exit criteria: What must be achieved for UAT sign-off? (All UAT test scenarios executed, all blocking defects resolved, business owner approval.)

46. Create the UAT test schedule: How many days? Which users test which modules? How are defects reported?
47. Prepare the UAT environment: Production-equivalent configuration, realistic test data, user accounts and permissions set up.

6.3 Writing UAT Test Scenarios

UAT test scenarios are written in business language — not technical language. They describe real user journeys in the context of actual business processes. A bank would write UAT scenarios around 'process a mortgage application', not 'click the Submit button'.

UAT Scenario Format

Scenario: Process a new customer mortgage application

Role: Relationship Manager

Pre-conditions: Customer record exists, supporting documents uploaded

Steps:

1. Login as Relationship Manager
2. Search for customer by National ID
3. Open existing customer record
4. Select 'New Mortgage Application' from the action menu
5. Complete all mandatory fields and submit

Expected Result: Application status changes to 'SUBMITTED', customer receives email confirmation, application appears in Branch Manager's queue

Business Rule: Application number must be generated in format MORT-YYYY-NNNNNN

6.4 Coordinating UAT Participants

Business users are not professional testers. Your role as QA is to make the UAT process as clear and frictionless as possible for them. Confusion or friction leads to poor UAT coverage and unreliable results.

- Brief users before UAT starts: Explain the purpose, schedule, how to report defects, and what a successful UAT outcome looks like.
- Assign scenarios to users: Match scenarios to users who work in those business processes daily.
- Provide a simple defect template: Use a Word form or JIRA intake form — business users cannot navigate a full JIRA workflow.
- Schedule daily check-ins: A 15-minute daily call during UAT catches blockers early and keeps momentum.
- Track progress visibly: A shared spreadsheet with scenario status (Not Started, In Progress, Passed, Failed, Blocked) gives everyone visibility.

In Practice

The most common UAT failure is treating it like a formality after system testing. Business users skip scenarios, rubber-stamp sign-off, and critical usability issues reach production. Treat UAT as a real test

phase with real ownership. Engage business users early, make it easy for them to report issues, and never allow sign-off without evidence that all scenarios were executed.

6.5 UAT Defect Management

UAT defects are different from system testing defects. They are often about business process gaps, missing functionality, or workflow mismatches rather than technical bugs. Triage them carefully.

UAT Defect Type	Example	Action
Blocking defect	Application cannot be submitted — button disabled	Fix before UAT sign-off
Business logic gap	Discount not applied for loyalty tier customers	Fix before sign-off; may require business rule clarification
Usability issue	Date format is MM/DD/YYYY but users expect DD/MM/YYYY	Fix in next sprint; document as known issue
Scope creep / new requirement	'Can we also add X feature?' request during UAT	Log as new requirement; do NOT add to current release
Training gap	User does not know how to use the feature	Address through user guide and training, not a code fix

6.6 UAT Sign-Off Process

Sign-off is the formal business acceptance that the software is ready for production. It is a business decision, not a technical one. The QA Engineer facilitates the sign-off process but does not own it.

48. Prepare a UAT completion report: Summary of scenarios executed, defects found and status, outstanding risks.
49. Conduct a sign-off meeting: Walk stakeholders through the completion report and outstanding defects.
50. Address blockers: Any blocking defects must be resolved (or explicitly deferred with sign-off) before sign-off.
51. Obtain written sign-off: An email, JIRA approval, or signed document from the designated business authority.
52. Archive the evidence: Store UAT test results, defect logs, and sign-off confirmation for audit purposes.

6.7 Best Practices

- Start UAT planning two sprints early — UAT environment setup, data preparation, and user scheduling all take time.
- Never skip entry criteria — deploying an unstable build to UAT demoralises business users and destroys confidence in the release.

- Keep scope tight — new requirements during UAT are a sign that requirements were not finalised before development. Resist scope creep.
- Document all decisions in writing — especially when a business user agrees to defer a defect to the next release.
- Post-UAT retrospective: Review what went well and what could be improved for the next UAT cycle.

6.8 Common Mistakes

- Starting UAT without stable software — if the build is still being fixed during UAT, you cannot get reliable results.
- No defined sign-off authority — UAT drags on indefinitely when no one knows who has final authority to say 'go'.
- Asking QA engineers to play the role of business users — QA knows the system too well; only real users find usability and workflow gaps.
- Treating UAT sign-off as automatic — sign-off must be earned by evidence of completed testing, not assumed.

Warning

If a business user wants to sign off UAT without completing all scenarios, do not accept it. Document that the scenarios were not completed, the risk this creates, and obtain written acknowledgement from the business that they accept this risk. 'Sign-off under pressure' without full coverage is a liability for the entire team.

6.9 Exercises

53. Write 5 UAT test scenarios for an online banking money transfer feature. Use the business scenario format with role, pre-conditions, steps, expected result, and business rules.
54. Create a UAT coordinator checklist covering all activities from 'UAT kick-off' to 'sign-off obtained'. Include at least 20 items.
55. Design UAT entry and exit criteria for a retail point-of-sale system replacing a legacy system.
56. You receive 8 UAT defects. Classify each as: blocking, business logic gap, usability issue, scope creep, or training gap. For each, define the appropriate action.
57. Draft a UAT completion report for a fictional project. Include: scenarios executed, pass/fail summary, defect status, outstanding risks, and sign-off recommendation.

Quick Quiz

1. What is the key difference between system testing and UAT?
2. Who should perform UAT — QA engineers or business users? Why?
3. What is UAT entry criteria and why is it important?
4. How would you handle a business user who wants to add new features during UAT?
5. What must be included in a formal UAT sign-off to protect the project team?

Interview Questions

1. Describe how you have coordinated a UAT cycle from planning to sign-off.
2. How would you handle a situation where business users are not participating in scheduled UAT sessions?
3. What do you do when a key business stakeholder wants to sign off UAT without completing all test scenarios?
4. How do you distinguish between a UAT defect that must be fixed before release and one that can be deferred?

Chapter Summary

• UAT verifies business fitness — that software works for real users in real business processes.
• Plan UAT at least two sprints in advance — environment, data, and user coordination take time.
• UAT scenarios are written in business language, not technical language.
• QA facilitates UAT but business users are the testers; their perspective reveals workflow and usability gaps QA cannot.
• Sign-off must be evidence-based, in writing, from a designated business authority.

Chapter 7 - Advanced Exploratory and Session-Based Testing

↑ Back to Index

Exploratory testing is simultaneously the most skilled and most misunderstood testing technique. Done poorly, it looks like random clicking with no structure. Done well, it is the most powerful defect-finding tool in a QA engineer's arsenal — uncovering issues that no scripted test case would ever find because nobody thought to write them.

At Level 2, you move beyond basic ad-hoc exploration to structured session-based exploratory testing with charters, heuristics, and session notes. This chapter teaches you to explore software systematically and produce evidence of your work.

7.1 What Makes Exploratory Testing Different

Scripted Testing	Exploratory Testing
Test cases written before execution	Test design and execution happen simultaneously
Follows a predetermined path	Adapts based on what is observed
Best for: verifying known behaviour	Best for: discovering unknown behaviour
Repeatable by anyone with the test case	Requires skill, domain knowledge, and intuition
Good for: regression, compliance, sign-off	Good for: new features, edge cases, usability issues

7.2 Session-Based Test Management (SBTM)

Session-Based Test Management structures exploratory testing into time-boxed sessions with a clear charter. Each session has a defined mission, a defined time limit, and produces documented session notes. This makes exploratory testing auditable and measurable.

Session Charter Format
Charter: Explore the user profile editing feature focusing on concurrent edit conflicts
Tester: [Your name]
Session Duration: 90 minutes
Area: User Profile — /settings/profile
Environment: Staging v2.4.1
Start Time: 10:00 | End Time: 11:30

Mission: Investigate what happens when two browser tabs have the same profile open and both submit changes simultaneously.

The charter defines the boundary of your session. You are free to follow any path within that boundary but should resist the temptation to wander outside it — save new areas for their own sessions.

7.3 Exploratory Testing Heuristics

Heuristics are rules of thumb that guide exploration toward productive areas. The most widely used framework is the mnemonic SFDIPOT (or 'San Francisco Depot'), created by James Bach.

Letter	Meaning	Questions to Ask
S	Structure	What is the system made of? What are all the inputs and outputs?
F	Function	What does it do? Test each function, including functions within functions.
D	Data	What data does it process? Test with boundary values, null, Unicode, huge inputs.
I	Interfaces	How does it connect to other systems? Test all integration points and APIs.
P	Platform	What OS, browser, device does it run on? Test all supported platforms.
O	Operations	How will it be used in production? Simulate real user workflows and volumes.
T	Time	How does it behave over time? Test with slow networks, session timeouts, date boundaries.

7.4 Bug Hunting Techniques

Experienced exploratory testers have a toolkit of strategies for finding defects that scripted tests miss:

- Follow the data: Trace a record through every system it touches. Create a user, check it in the database, look for it in reports, delete it, check it is gone from all places.
- Stress the boundaries: Test what happens at the extreme edges of every field, limit, and threshold.
- Break the workflow: Try operations in the wrong order. Log out mid-process. Press Back after submitting. Refresh during a multi-step form.
- Test concurrent actions: Open two tabs and perform the same action simultaneously. Try to create duplicate records.
- Test permissions negatively: Try to access pages and actions you are not authorised for. Test that lower-privilege users cannot elevate their access.
- Test with real data: Use the kinds of names, addresses, and content that real users have — including special characters, long text, non-Latin scripts, and emoji.

How It Works Internally

The best exploratory testing bugs are found when the tester thinks like an adversarial user — someone who is confused, impatient, creative, or malicious. The majority of production defects are caused by users doing things the developers did not anticipate. Exploratory testing simulates this unpredictability in a controlled, documented way.

7.5 Writing Session Notes

Session notes are the audit trail of exploratory testing. They record what you did, what you found, and what questions arose. They do not need to be perfectly formatted — they need to be honest and useful.

Session Notes Format

Session: Explore user profile concurrent edits
Time spent: 85 minutes
Test coverage areas: Save button, concurrent tab behaviour, session token handling

Findings:
- DEFECT: Saving from Tab 2 after Tab 1 saved silently overwrites Tab 1's changes with no conflict warning [logged as BUG-441]
- DEFECT: Profile photo upload fails silently if file > 4MB; no error message shown [logged as BUG-442]
- OBSERVATION: The avatar initials use first name only — is middle name expected? Flagged to PO
- NOTE: The 'Discard changes' button is greyed out after navigating away and back — unclear if by design

Areas not covered: Mobile view, password change tab (separate session needed)

7.6 Measuring Exploratory Testing

A common criticism of exploratory testing is that it is unmeasurable. SBTM addresses this directly:

Metric	How to Measure
Sessions completed	Count of completed charters for a feature or sprint
Test coverage by area	Which SFDIPOT dimensions were covered per feature?
Defect yield per session	Defects found ÷ sessions completed (aim for > 1 per session)
Time ratio	Testing time vs bug investigation time vs setup time per session
Charter coverage	Planned sessions vs completed sessions (coverage of intended exploration)

7.7 Best Practices

- Always start an exploratory session with a charter — unstructured exploration without a mission produces shallow results.
- Time-box sessions to 60–120 minutes; longer sessions suffer from fatigue and reduced defect-finding effectiveness.
- Take screenshots and notes during the session — memory is unreliable; document findings immediately.
- Debrief after each session: review your notes, raise defects, and define charters for follow-up sessions.
- Combine exploratory and scripted testing: use scripted tests for regression and compliance; use exploratory for new features and complex interactions.

7.8 Common Mistakes

- Exploring without a charter — leads to shallow, duplicated, and undocumented testing.
- Not raising defects during the session — relying on memory leads to missed defects.
- Treating exploratory testing as 'play time' — it is a professional skill that requires preparation, discipline, and documentation.
- Only exploring happy paths — the whole point of exploration is to go beyond the scripted happy path.

Warning

Exploratory testing is NOT a replacement for structured regression testing. It is a complement. Use scripts to verify known requirements; use exploration to discover unknown failures. Teams that abandon regression for pure exploration tend to have high rates of escaped regressions.

7.9 Exercises

58. Write 5 exploratory test charters for a ride-sharing app. Each charter should have a clear mission, area, and 90-minute time-box.
59. Conduct a 30-minute exploratory session on any web application you use regularly. Document your charter, session notes, and at least 3 observations or defects.
60. Apply the SFDIPOT heuristic to a login page. For each dimension (S, F, D, I, P, O, T), write at least 3 test ideas.
61. Review the session notes format above and write a template you would use on your next project.
62. Design a session-based test plan for a 3-day UAT with 4 testers and 5 major feature areas. Assign charters to testers and define coverage goals.

Quick Quiz

1. What does SBTM stand for and what problem does it solve in exploratory testing?
2. What is a session charter and what must it contain?

3. Explain the SFDIPOT heuristic and give one test idea for each dimension for a search feature.
4. How do you measure the effectiveness of an exploratory testing session?
5. What is the difference between exploratory testing and ad-hoc testing?

Interview Questions

1. How do you structure and document your exploratory testing sessions?
2. Give an example of a critical bug you found (or would find) through exploratory testing that scripted tests would miss.
3. How do you balance exploratory testing with scripted regression testing in a sprint?
4. Walk me through how you would explore a complex checkout flow for the first time.

Chapter Summary

• Session-based exploratory testing structures exploration with charters, time-boxes, and session notes.
• SFDIPOT heuristic guides exploration across Structure, Function, Data, Interfaces, Platform, Operations, and Time.
• Bug hunting techniques target the gaps between what developers anticipated and what users actually do.
• Session notes provide the audit trail that makes exploratory testing credible and measurable.
• Exploratory testing complements scripted testing — it is not a replacement for regression coverage.

Chapter 8 - Cross-Browser, Cross-Platform and Compatibility Testing

↑ Back to Index

The same application running on Chrome on a Windows laptop and Safari on an iPhone can behave very differently. CSS renders differently. JavaScript behaves differently. Screen sizes change layouts. Touch interactions differ from mouse interactions. Cross-browser and compatibility testing ensures your application works correctly for all users, regardless of their device, browser, or operating system.

In a world where users access products on hundreds of device and browser combinations, you cannot test every combination. This chapter teaches you to build a practical compatibility test strategy that maximises coverage for the combinations that matter most to your users.

8.1 The Compatibility Testing Challenge

How It Works Internally

Web browsers are not identical. Each browser has its own JavaScript engine (V8 for Chrome, SpiderMonkey for Firefox, JavaScriptCore for Safari) and its own CSS rendering engine. Features available in Chrome may not be available in older Safari versions. CSS Flexbox and Grid behave subtly differently across browsers. Mobile browsers have additional constraints: touch events, viewport scaling, limited memory, and network variability.

Browser / Platform	Global Market Share (2026 approx.)	Testing Priority
Chrome (Desktop)	~63%	Critical — always test
Safari (Mobile/Desktop)	~19%	Critical — largest mobile browser
Edge (Desktop)	~5%	High — enterprise users
Firefox (Desktop)	~3%	Medium — developer/privacy users
Samsung Internet	~2.5%	Medium — large Android share
Older Chrome/Safari	~2%	Low — test if analytics show traffic

8.2 Building a Browser Test Matrix

A browser test matrix defines exactly which browser/OS combinations you will test, and at what level of depth. Define this at project start based on your analytics data.

Browser	OS	Version	Test Level	Rationale
Chrome	Windows 11	Latest	Full regression	Primary desktop browser

Browser	OS	Version	Test Level	Rationale
Safari	iOS 17	Latest	Full regression	Dominant mobile browser
Chrome	Android 14	Latest	Full regression	Dominant mobile Android
Edge	Windows 11	Latest	Core journeys	Enterprise primary
Firefox	Windows 11	Latest	Core journeys	Developer audience
Safari	macOS Sonoma	Latest	Core journeys	Mac user base

8.3 What to Test in Cross-Browser Testing

Don't repeat your full test suite for every browser — that would be impractical. Instead, focus on areas most likely to differ across browsers.

- Layout and rendering: Do pages display correctly? Are there layout breaks on different screen widths?
- CSS compatibility: Are fonts, colours, animations, and responsive layouts consistent?
- Form behaviour: Do form validations, date pickers, and dropdowns work correctly?
- JavaScript-dependent features: Are modals, dynamic content loading, and interactive components functional?
- Performance: Does the page load in acceptable time on the target browser? (Safari on mobile is often slower than Chrome on desktop.)
- Touch interactions: On mobile, do tap targets work? Are menus swipeable? Are forms usable on a small screen?
- File upload and download: Browsers handle file operations differently, especially on mobile.
- Authentication: Session handling and cookie behaviour can differ between browsers.

In Practice

Use your application analytics (Google Analytics or Mixpanel) to identify your users' actual browser and device distribution. A B2B enterprise tool used entirely by Windows/Chrome users needs very different compatibility coverage than a consumer app with 70% mobile traffic. Build your test matrix around your real user base, not theoretical coverage.

8.4 Responsive Design Testing

Responsive design testing verifies that your application adapts correctly to different screen sizes. Test at standard breakpoints used in your design system.

Breakpoint	Width	Target Devices
Mobile Small	< 375px	Small iPhones, compact Android
Mobile	375–430px	iPhone 14, standard Android
Tablet Portrait	768px	iPad, Android tablets
Tablet Landscape / Small Desktop	1024px	iPad landscape, small laptops
Desktop	1280–1440px	Standard laptop and desktop monitors
Wide Desktop	> 1920px	Large monitors and 4K displays

8.5 Tools for Cross-Browser Testing

Tool	Type	Best Used For
BrowserStack	Cloud real device farm	Testing on real devices without owning them; manual and automated
Sauce Labs	Cloud device farm	CI/CD integration for automated cross-browser suites
Chrome DevTools	Built-in browser tool	Quick responsive design checking; device emulation; console errors
Firefox Developer Edition	Browser	CSS Grid and layout debugging
CrossBrowserTesting (CBT)	Cloud tool	Screenshot testing across 2,000+ combinations
Playwright	Automation framework	Cross-browser automated tests in Chromium, Firefox, and WebKit

8.6 OS and Device Compatibility

Beyond browsers, you must consider the operating system and device hardware, particularly for mobile applications.

- iOS versions: Apple users update quickly, but some users stay on iOS 15–16. Define minimum iOS version based on analytics.
- Android fragmentation: Android has thousands of device models with varied screen sizes, processors, and OS versions. Focus on the most common devices in your user base.
- Screen density: Retina displays (2x/3x) on Apple and high-DPI displays on Android can expose image and icon quality issues.

- Keyboard and IME: International users type in different languages with different input methods — test with non-Latin character inputs.
- Accessibility modes: Test with Dark Mode, Large Text, and High Contrast modes enabled.

8.7 Best Practices

- Define your browser matrix at project start and review it each quarter based on analytics.
- Use the Chrome DevTools device emulator for quick checks during development — it is not a substitute for real device testing but catches obvious issues fast.
- Always test on real devices for mobile — emulators miss touch accuracy, performance, and rendering differences.
- Log browser/device details with every defect raised during compatibility testing — a bug that only occurs on Safari/iOS is uninvestigatable without this information.
- Integrate cross-browser tests into your CI pipeline using Playwright or Selenium WebDriver with BrowserStack/Sauce Labs.

8.8 Common Mistakes

- Treating cross-browser testing as optional — browser differences are real and affect real users; this is not edge-case testing.
- Only testing in Chrome — developers build and test in Chrome, so Chrome-only bugs are common.
- Not testing on real mobile devices — the emulator in Chrome DevTools does not accurately reproduce Safari/WebKit rendering or touch behaviour.
- Ignoring older OS versions — enterprise users may be on Windows 10 with Edge 105; consumer users may have older iPhones running iOS 15.

Warning

Safari on iOS has historically been the most common source of cross-browser defects for web applications. Apple strictly enforces the use of WebKit as the rendering engine for all browsers on iOS, which means Chrome on iPhone uses WebKit, not Blink. Always test on a real iPhone — Chrome DevTools emulation does not reproduce Safari/WebKit behaviour.

8.9 Exercises

63. Build a browser test matrix for a travel booking website. Use publicly available browser market share data to justify your choices. Define test depth for each combination.
64. Use Chrome DevTools to test any website at 5 different viewport widths. Document the responsive behaviour and identify any layout breaks.
65. Create a cross-browser defect template that captures: browser, browser version, OS, OS version, device, steps to reproduce, expected result, and actual result.
66. Research the browser and OS distribution for any application you work on. How does the real distribution compare to global averages? How does this change your test matrix?

67. Design a CI/CD compatible cross-browser test strategy using Playwright. Specify which browser/OS combinations to run in CI and which to run in scheduled nightly builds.

Quick Quiz

1. Why does the same web application sometimes behave differently in Chrome and Safari?
2. What is a browser test matrix and what data should drive its construction?
3. Why is testing on Chrome DevTools device emulator not sufficient for mobile testing?
4. What is the significance of WebKit on iOS devices for cross-browser testing?
5. How would you integrate cross-browser testing into a CI/CD pipeline?

Interview Questions

1. How do you decide which browsers and devices to test on for a new project?
2. Describe a cross-browser defect you found (or would look for) that affected a specific browser only.
3. How do you balance thorough cross-browser coverage with sprint time constraints?
4. What tools and techniques do you use for responsive design testing?

Chapter Summary

• Browsers and devices render and behave differently — compatibility testing is essential, not optional.
• Build your test matrix based on real user analytics, not theoretical coverage.
• Focus cross-browser testing on layout, CSS, JavaScript features, forms, and touch interactions.
• Always test on real mobile devices — emulators cannot reproduce WebKit/Safari rendering accurately.
• Use tools like BrowserStack, Playwright, and Chrome DevTools to scale coverage across your target matrix.

Capstone Project: Level 2 QA Engagement Simulation

↑ Back to Index

Project Overview

You are the QA Engineer for 'SwiftPay' — a fintech startup launching a personal payments app. The app allows users to register, link a bank account, send money to contacts, and view transaction history. You are 3 weeks into a 4-week Agile project. Sprint 3 begins Monday.

Your Task

Complete all five deliverables below across one simulated sprint. This project brings together every skill from Phase 3: Agile testing, API testing, database verification, metrics, risk-based prioritisation, UAT preparation, exploratory testing, and cross-browser coverage.

Deliverables

68. Sprint Test Plan: Write a Sprint 3 test plan. Features in scope: Bank Account Linking (new), Send Money Flow (new), Transaction History (change). Apply risk-based prioritisation with a risk matrix for all three features. Define quality gates.
69. API Test Collection: Design a Postman test collection for the POST /transfer endpoint. Include: authentication test, happy path (valid transfer), negative tests (insufficient funds, invalid recipient, amount > daily limit), and schema validation. Write test scripts with assertions.
70. Database Verification Queries: For a successful money transfer, write SQL queries to verify: sender balance decremented, receiver balance incremented, transaction record created with correct fields, audit log entry created, no duplicate transaction records.
71. Sprint Metrics Report: At sprint close, your results are: 145/160 planned test cases executed, 131 passed, 14 failed, 3 blocked, 22 defects found (P1: 1, P2: 5, P3: 16). Calculate all key metrics and write a go/no-go recommendation with justification.
72. Exploratory Testing Session: Write 3 SBTM session charters for the Send Money flow. Execute one 45-minute session and produce session notes including all findings, questions, and follow-up charter ideas.

Chapter Summary

- Risk matrix + sprint test plan demonstrates strategic QA planning.
- API test collection demonstrates Postman/REST testing competency.
- SQL verification queries demonstrate database testing capability.
- Metrics report with go/no-go demonstrates reporting and stakeholder communication skills.
- SBTM session demonstrates structured exploratory testing discipline.

Case Study: The API Regression That Cost 48 Hours

↑ Back to Index

The Situation

Priya is a QA Engineer at a mid-sized logistics company. Her team uses an Agile delivery model with 2-week sprints. In Sprint 14, the development team made what they described as a 'minor refactoring' of the order management API — no functional changes, just internal code cleanup. The sprint had 6 user stories and a tight deadline.

Because the change was labelled 'refactoring', the risk was assessed as Low and the order management API tests were moved out of scope to save time. The sprint was delivered on schedule. Three days after release, the customer service team began receiving calls — international orders were failing silently. No error was shown to users; orders simply disappeared.

What Went Wrong

- The 'minor refactoring' changed the internal data model for international orders — a field was renamed from destination_country_code to country_code. The front-end sent destination_country_code; the API now silently ignored it.
- No API regression tests were run. There was no automated contract test to detect the field name change.
- The database was not checked post-deployment — a database test would have caught the missing country_code values immediately.
- The risk assessment was done by the development team, not QA. QA was not consulted.

The Impact

1,247 international orders over 3 days were created without a country code. These orders could not be routed to the correct fulfilment centre. The operations team spent 48 hours manually identifying and re-routing affected orders. Three enterprise customers threatened to terminate their contracts. The reputational and financial impact was estimated at $200,000.

Lessons Applied from Phase 3

Lesson	Application
API regression is mandatory	All API tests must run for any code change, regardless of description
QA owns risk assessment	QA must be consulted before labelling any change as 'low risk'
Database verification post-deploy	A post-deployment DB check would have caught the issue within minutes
Contract testing	Implement contract tests (e.g. Pact) to catch field name changes automatically

Lesson	Application
Code change = regression trigger	Any code change — even refactoring — triggers a targeted regression pass

After this incident, Priya implemented a mandatory API regression suite in the CI pipeline. Every merge to main runs the full API test collection. The database is checked by a monitoring query 5 minutes after every production deployment. No similar issue has occurred in the 18 months since.

Mock Assessment: Phase 3 — Level 2 QA Engineer

↑ Back to Index

Section A: Multiple Choice (20 marks)

73. In Scrum, when should a QA engineer first review a user story? a) When the developer marks it as done b) During sprint planning and Three Amigos c) On the last day of the sprint d) During the sprint retrospective
74. A POST /users API returns a 409 status code. This typically means: a) The request was successful b) The resource was not found c) A duplicate resource already exists d) The user is not authenticated
75. Which SQL clause is used to verify referential integrity by checking for orphaned records? a) GROUP BY b) LEFT JOIN ... WHERE right.id IS NULL c) HAVING COUNT(*) > 1 d) ORDER BY
76. Risk Priority Number (RPN) is calculated as: a) Likelihood + Impact b) Likelihood × Impact c) Impact ÷ Likelihood d) (Likelihood + Impact) ÷ 2
77. A Defect Removal Efficiency of 98% means: a) 98% of defects were fixed in QA b) 98% of all defects were found before production c) 2% of test cases passed d) 98 defects were removed from the backlog

Section B: Short Answer (40 marks)

78. Describe the Three Amigos meeting. Who attends, what is discussed, and what is the output? (8 marks)
79. You have a Postman collection with 30 API tests. Describe how you would integrate this collection into a CI/CD pipeline using Newman. What environment variables would you use? (8 marks)
80. Write 4 SQL queries to verify a bank transfer: (a) sender balance reduced, (b) receiver balance increased, (c) transaction record created, (d) no duplicate transactions. (10 marks)
81. A sprint ends with 25% of tests unexecuted due to a critical blocker. Using your knowledge of RBT and quality gates, write a release recommendation. (8 marks)
82. Write a session charter for an exploratory test of a password change feature, using the SBTM format. Include charter, area, duration, and mission. (6 marks)

Section C: Scenario-Based (40 marks)

83. A new payment gateway integration has been added to your e-commerce platform. The developer says it is 'fully tested' at the unit level. Design a complete QA test approach covering API testing, database verification, cross-browser testing, risk assessment, and UAT. What would you test and why? (20 marks)
84. Your sprint has 8 features to test but only 5 days of QA time. Using Risk-Based Testing principles, describe how you would prioritise, what you would defer, how you would document the residual risk, and what sign-off you would require before release. (20 marks)

Quick Reference: Level 2 QA Engineer Cheat Sheet

↑ Back to Index

Agile Ceremonies — QA Role

Ceremony	QA Focus
Sprint Planning	Raise testability concerns, estimate QA effort
Three Amigos	Define acceptance criteria and test scenarios with PO + Dev
Daily Standup	Report progress, blockers, dependency risks
Sprint Review	Present test coverage and defect status
Retrospective	Identify process improvements for QA

HTTP Status Codes — QA Quick Reference

Code	Meaning and Test Implication
200 OK	GET/PUT success — verify response body
201 Created	POST success — verify location header and body
204 No Content	DELETE success — verify resource is gone
400 Bad Request	Invalid input — verify all validation rules trigger 400
401 Unauthorized	Missing/invalid token — test with no token and expired token
403 Forbidden	Valid token, insufficient role — test role boundaries
404 Not Found	Resource doesn't exist — test with invalid IDs
409 Conflict	Duplicate resource — test duplicate creation
500 Internal Server Error	Server error — log and escalate; do not ignore

Key QA Metrics Formulas

Metric	Formula
Test Execution Rate	Executed ÷ Planned × 100
Test Pass Rate	Passed ÷ Executed × 100
Defect Density	Defects ÷ Test Cases × 100

Metric	Formula
DRE	QA Defects ÷ (QA Defects + Production Defects) × 100
Escaped Defect Rate	Production Defects ÷ Total Defects × 100
RPN (Risk)	Likelihood × Impact

Risk Priority Matrix

RPN Score	Test Priority
20–25	Critical — full test suite + exploratory
12–19	High — full regression + edge cases
6–11	Medium — standard test suite
1–5	Low — smoke test only

SQL Quick Reference for QA

```
-- Count rows by status
SELECT status, COUNT(*) FROM orders GROUP BY status;

-- Find orphaned records (referential integrity)
SELECT * FROM orders WHERE customer_id NOT IN (SELECT id FROM users);

-- Find duplicates
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Verify field not null
SELECT COUNT(*) FROM transactions WHERE transfer_reference IS NULL;

-- Verify soft delete
SELECT id, status, deleted_at FROM users WHERE id = ?;
```

SBTM Session Charter Template

Field	Value
Charter (Mission)	Explore [feature] focusing on [risk area]
Area	URL / module / feature path
Duration	60–90 minutes
Tester	Your name

Field	Value
Environment	Env name + build version
Entry Condition	What must be true before you start?

Quality Gate Checklist — Pre-Release

- Test Execution Rate >= 95%
- Test Pass Rate >= 90%
- Zero P1 (Critical) defects open
- P2 (High) defects <= 2 with documented workarounds
- All regression critical path tests passing
- UAT sign-off obtained from business owner
- API regression suite run and passed
- Database post-deployment verification run
- Cross-browser smoke test on Chrome + Safari passed
- Performance benchmarks within agreed SLA